

Inheritance vs Composition

Spring tells us that it can easily manage the dependency between the class, but you have to follow its given rules and regulation through which it can help.

- Rules
- Design Pattern
- Strategy Design Pattern

Design Pattern:

There are repeated type of problem which experts had seen and observed, and provided the best set of solution for it that should be applied for that particular situation. This is known as a design pattern.

- Strategy Design Pattern
- It is provided by the set of experts.

Core Principles

- Favor composition over inheritance
- Design to interface, never design to concrete classes
- Code should be open for extension and close for modification

In how many ways we can use one class's functionality into another class?

- There are two ways:
- Inheritance
 - Composition

Inheritance is simply telling us that we can use one functionality of another class by using the extends keyword.

Example :

<pre>class A{ int a=10; public int m1(){ return a; } }</pre>	<pre>class B extends A { public int m2(){ return a; } }</pre>
--	---

Composition is when we use one functionality of a class into another class by writing the attribute of one class into another.

<pre>class A{ int a=10; public int m1(){ return a; } }</pre>	<pre>class B { A ref; public int m2(){ return ref.a; } }</pre>
---	--

- Inheritance is known as an “is-a” relation
- Composition is known as a “has-a” relation

Can you prove it?

Inheritance and Composition Example
Point to be noticed in the below set of code:

Inheritance

When we use the extends keyword, then without creating the object of the A01 class, we can get all the functionality of the A01 class through B01.
So we can say that if we are able to use all the functionality of the parent class through the child class — without reaching directly to the parent class — or we can say the child is the same as the parent, then it's an “is-a” relation. Inheritance is the example of an “is-a” relation.

Composition

But if we see in the case of composition, we cannot access all the functionality of the A01 class through the B01 class, because B01 is not the same as A01 — meaning we have not used the extends keyword which inherits all the properties of the parent into the child. Instead, B01 class has A01 as a reference, and through that reference only we can access the functionality. Because of this, we can call it a “has-a” relation.

```
class A01{
    public int m1() {
        return 10;
    }

    public int m2() {
        return 20;
    }

    public void m3() {
        System.out.println("Hello ");
    }
}

class B01 {
    A01 REF; //extends A01 {
    public int m4() {
        REF.m2();
        return 100;
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        /* A01 a=new A01();
        a.m1();
        a.m2(); a.m3();
        a.m4(); */
        B01 b=new B01();
        b.m1();
        b.m2();
        b.m3();
        b.m4();
    }
}
```

- When We Can Go For Inheritance
- When you want all the functionality of the parent class into the child, then you can go for inheritance.
 - When you can say that without reaching to the parent class, you are able to get the functionality of the parent class through the child, then we can go for inheritance.

Example:

RBI Bank	SBI Bank
- amount	
- loan	have the same functionality which RBI has
- deposit	
- rules	

- When We Can Go For Composition
- When you want only a few functionalities of the parent class into the child class, then we can say to use composition.

- Favor Composition Over Inheritance
- Inheritance has some issues, that's why we go for composition.
 - Most of the real time use cases can be solved through composition.
 - Cannot support multiple inheritance.
 - By using inheritance, our classes become fragile.

Example
Most of the real time use cases can be solved through composition.
package com.task.demo;

```
class NaukriService{
    public String uploadResume(String resume) {
        return "Resume uploaded successfully";
    }

    public boolean IsHiring() {
        return false;
    }

    public String HRMail() {
        if(IsHiring()) {
            return "abc@123alphabeta.com";
        }
        else {
            return "We will connect you sortly";
        }
    }
}

class NaukriApp {
    NaukriService ref;

    public String uploadYourResume() {
        return ref.HRMail();
    }
    /*public String uploadResume(String resume) {
        // TODO Auto-generated method stub
        return super.uploadResume(resume);
    }
    @Override
    public boolean IsHiring() {
        // TODO Auto-generated method stub
        return true;
    }

    @Override
    public String HRMail() {
        if(IsHiring()) {
            return "abc@123alphabeta.com";
        }
        else {
            return "We will connect you sortly";
        }
    }
    */
}

public class NaukriUser {
    public static void main(String[] args) {
        NaukriApp app =new NaukriApp();
        // System.out.println(app.HRMail());
        app.uploadYourResume();
    }
}
```